Practico 1
Expresiones Cuantificadas

2

a) $\langle \forall i : 0 \leq i < \#xs : xs.i > 0 \rangle$
Todo elemento de xs es mayor que 0

b) $\langle \exists i : 0 \leq i < \#xs : xs.i = x \rangle$
Existe un elemento en la lista que es igual a x

c) $\langle \forall i : 0 \leq i < \#xs : \langle \exists j : 0 \leq j < \#ys : xs.i = ys.j \rangle \rangle$
Para todo elemento de la lista xs existe un elemento igual en la lista ys

d) $\langle \forall i : 0 \leq i < \#xs - 1 : xs.i = xs.(i+1) \rangle$
para todo elemento de la lista es igual a su siguiente

3

Para cada uno de los ıtems del ejercicio anterior, evalúe la formula en las siguientes listas:

a) xs =[−5,−3,4,8]

$\langle \forall i : 0 \leq i < \#xs : xs.i > 0 \rangle$
≡ { calculo rango sabiendo que #xs = 4 }
$\langle \forall i : i \in \{0,1,2,3\} : xs.i > 0 \rangle$
≡ { aplicó el termino a cada elemento del rango }
$(xs.0 > 0) \wedge (xs.1 > 0) \wedge (xs.2 > 0) \wedge (xs.3 > 0)$
≡ { evaluó las indexaciones con xs = [−5,−3,4,8] }
$(-5 > 0) \wedge (-3 > 0) \wedge (4 > 0) \wedge (8 > 0)$
≡ { evaluó las desigualdades }
False ∧ False ∧ True ∧ True
≡ { resuelvo las conjunciones }
False
NO CUMPLE

4) Escriba fórmulas para las siguientes expresiones en lenguaje natural.

a) Todos los elementos de xs ocurren en ys

$\langle \forall i : 0 \leq i < \#xs: \langle \exists j : 0 \leq j < \#ys : xs.i = ys.j \rangle \rangle$

b) Todos los elementos de xs ocurren en ys en la misma posición.

c) Todos los elementos de xs e ys son iguales (¡ojo! ¡sujeta a interpretación!).

$\langle \forall i : 0 \leq i < \#xs: \langle \forall j : 0 \leq j < \#ys : xs.i = ys.j \rangle \rangle$

5)

Suponiendo que f : A → Bool es una función fija cualquiera, y xs : [A]. Caracterice con una cuantificación la siguiente funci´on recursiva:

algunof : [A] → Bool
algunof.[ ] = False
algunof.(x ▷ xs) = f.x ∨ algunof.xs

$\langle \exists x : 0 < x < \#xs : f(xs.x) \rangle$

explicación: La función **algunof** verifica si existe al menos un índice x en la lista xs para el cual **f(xs.x)** es **True**

6)

Defina recursivamente una funcion todos : [Bool] → Bool que verifica que todos los elementos de una lista son True, es decir, que satisface la siguiente especificación:

todos.xs ≡ $\langle \forall i : 0 \leq i < \#xs : xs.i \rangle$

```
todos : : [Bool] → Bool
todos [ ] = True
todos (x:xs)  | x== False = False
              | Oterwise = todos xs
```

7)

Es enunciar no jodan

8. (Separacion de un termino) Demuestre los siguientes teoremas utiles para la materia.

Suponga que $\oplus$
es un cuantificador asociado a un operador gen´ erico $\oplus$, que es conmutativo y asociativo (asi como el $\forall$ es el cuantificador asociado a la conjunción $\wedge$) y n : Nat.

a) $\langle$ i : 0 ≤ i < n+1: T.i $\rangle$ = $\langle$ i : 0 ≤ i < n : T. i$\rangle$ $\oplus$ T.n

b) $\langle$ i :0 ≤ i < n+1 : T.i $\rangle$ = T.0 $\oplus$ $\langle$ i : 0≤i < n : T.(i+1) $\rangle$

A)

$\langle\oplus$i : 0 ≤ i < n+1: T.i $\rangle$ = $\langle\oplus$i : 0 ≤ i < n : T. i$\rangle$ $\oplus$ T.n
{Usare el lado izquierdo para desarrollar}
$\langle\oplus$i : 0 ≤ i < n+1: T.i $\rangle$
≡{Logica}
= $\langle\oplus$i :i = n v 0 ≤ i < n : T.i $\rangle$
≡ {Particion de Rango}
= $\langle\oplus$i : i = n : T.i $\rangle$ $\oplus$ $\langle\oplus$i : 0 ≤ i < n : T.i $\rangle$
≡ { Rango Unitario}
= T.n $\oplus$ $\langle\oplus$i : 0 ≤ i < n : T.i $\rangle$
≡{Conmutatividad}
= $\langle\oplus$i : 0 ≤ i < n : T.i $\rangle$ $\oplus$ T.n

B)

$\langle$ i :0 ≤ i < n+1 : T.i $\rangle$ = T.0 $\oplus$ $\langle$ i : 0≤i < n : T.(i+1) $\rangle$

{Usare el lado izquierdo para desarrollar}

$\langle\oplus$ i : 0 ≤ i < n+1 : T.i $\rangle$
≡ {logica}
= $\langle\oplus$i : i=0 v 1 ≤ i < n+1 : T.i $\rangle$
≡ {Particion de rango}
= $\langle\oplus$i : i=0 : T.i $\rangle$ $\oplus$ $\langle\oplus$ i : 1 ≤ i < n+1 : T.i $\rangle$
≡{Rango Unitario} {Cambio de variable i:= i+1}
= T.0 $\oplus$ $\langle\oplus$ i : 1 ≤ i+1 < n+1 : T.i+1 $\rangle$
≡{Aritmetica}
= T.0 $\oplus$ $\langle\oplus$i : 1 ≤ i+1-1 < n+1-1 : T.i+1 $\rangle$
≡{Aritmetica}
= T.0 $\oplus$ $\langle\oplus$i : 1 ≤ i < n : T.i+1 $\rangle$

listo

9) Defina recursivamente las siguientes funciones.

a) La funcion paratodo, que dada una lista de valores xs : [A] y un predicado T : A → Bool, determina si todos los elementos en xs hacen verdadero el predicado T, es decir:

paratodo : [A] → (A → Bool) → Bool

paratodo.xs.T ≡ ⟨∀i : 0 ≤ i < #xs : T.(xs.i)⟩

```
paratodo :: [A] -> (A -> Bool) -> Bool
paratodo [] _ = True  -- Caso base: una lista vacía satisface el predicado.
paratodo (x:xs) T
| T x == False = False  -- Si el predicado no se cumple para x, devolvemos False.
| otherwise = paratodo xs T  -- Si se cumple, continuamos con el resto de la lista.
```

b) La funci´on existe, que dada una lista de valores xs : [A] y un predicado T : A → Bool, determina si alg´ un elemento en xs hace verdadero el predicado T, es decir:

existe : [A] → (A → Bool) → Bool

existe.xs.T ≡ ⟨∃i : 0 ≤ i < #xs : T.(xs.i)⟩

```
existe : : [A] → (A → Bool) → Bool
existe  [] _ = False
existe (x:xs)  T | T x == True = True
                 | otherwise = existe xs T
```

c) La funcion sumatoria, que dada una lista de valores xs : [A] y una funcion T : A → Num (toma elementos de A y devuelve numeros), calcula la suma de la aplicacion de T a los elementos en xs es decir:

sumatoria : [A] → (A → Num) → Num
sumatoria.xs.T = ⟨ i : 0 ≤ i < #xs : T.(xs.i)⟩

```
sumatoria : : [A] → (A → Num) → Num
sumatoria [ ] _ = 0
sumatoria (x:xs) T = T x + sumatoria xs T
```

d) La función productoria, que dada una lista de valores xs : [A] y una función T : A → Num, calcula el producto de la aplicación de T a los elementos de xs, es decir:

productoria : : [A] → (A → Num) → Num
productoria.xs.T = ⟨ i : 0 ≤ i < #xs : T.(xs.i)⟩

```
productoria : : [A] → (A → Num) → Num
productoria [ ] _ = 1
productoria (x:xs) T = T x * productoria xs T
```

10)

```
cuantGen :: (b -> b -> b) -> b -> [a] -> (a -> b) -> b
cuantGen _ z [ ] _ = z  -- Caso base: lista vacía, devuelve el elemento neutro.
cuantGen op z (x:xs) T = op (T x) (cuantGen op z xs T)  -- Aplica el operador recursivamente.
```

12)

a) Podemos definir un cuantificador de conteo N utilizando la sumatoria:
⟨Ni : R.i : T.i⟩ = ⟨∑ i : R.i ^ T.i : 1⟩
a) Enuncie y demuestre las reglas de rango vacío, rango unitario y partición de rango para N.

RTA

Rango vacio

⟨Ni : False : T.i⟩ = 0
{Utilizamos la definicion de conteo}
⟨∑i : False ^ T.i : 1⟩ = 0
{Absorbente de la Conjuncion}
⟨∑i : False : 1⟩ = 0
{Rango vacio}
0 = 0

Rango Unitario
Me mato

Particion de Rango

⟨Ni : R.i v S.i : T.i⟩ = ⟨Ni : R.i : T.i⟩ + ⟨Ni : S.i : T.i⟩
{Agarramos el lado izq}

⟨Ni : R.i v S.i : T.i⟩
{Aplicamos la definición de conteo}
= ⟨∑i : (R.i v S.i) ^ T.i : 1⟩
{Distributiva de v con el ^}
= ⟨∑i : (R.i^ T.i) v (S.i^ T.i): 1⟩
{aplico partición de rango}
= ⟨∑i : (R.i^ T.i): 1⟩ + ⟨∑i : (S.i^ T.i): 1⟩
{Vuelvo a usar def de conteo}
=⟨Ni : R.i: T.i⟩ + ⟨Ni : S.i: T.i⟩

b) ni puta idea
c) ¿Que reglas de los cuantificadores generales no valen? ¿Por que?
No valen Regla del Termino, Termino constante, Distributivvidad, Anidado
no valen porque… no se porque

13)

a) $\langle\ \exists : k \in \mathbb{N} : n = 2\text{^}k\ \rangle$
b) $\langle \forall i : 0 \leq i < \#xs : xs.i \leq n \rangle \wedge \langle \exists j : 0 \leq j < \#xs : xs.j = n \rangle$
c) $\langle \prod i : 0 \leq i < \#xs \wedge \text{par}(xs.i) : xs.i \rangle$
d) $\langle \sum i : 0 \leq i < \#xs \wedge \text{par}.i : xs.i \rangle$

Practico 2

1)

1. Dado el **programa**

$$\begin{array}{|l} sum : [Num] \to Num \\ \hline sum.[\,] \doteq 0 \\ sum.(x \triangleright xs) \doteq x + sum.xs \end{array}$$

a) ¿Qué hace esta función? Escriba en lenguaje natural el **problema** que resuelve.
b) Escriba una **especificación** de la función con una expresión cuantificada.
c) **Verifique** que esta especificación vale para toda lista xs.

a) suma todos los elementos de la lista
b) sum.xs = ⟨∑x: 0 ≤ x < #xs : xs.x⟩
c) (Creo que es lo mismo)

2) Derivacion

sum.xs = ⟨∑x: 0 ≤ x < #xs : xs.x⟩

Caso Base xs= [ ]

sum.[ ] = ⟨∑x : 0 ≤ x < #[ ] : [ ]. x⟩
{Caso base de xs}
sum.[ ] = ⟨∑x: 0 ≤ x < 0 : xs.x⟩
{Logica}
sum.[ ] = ⟨∑x: False : xs.x⟩
{Rango Vacio}
sum.[ ] = 0

Caso Recursivo xs= x:xs

sum.x:xs = ⟨∑x: 0 ≤ x < #(x:xs) : (x:xs).x⟩
{Caso recursivo de #}
sum.x:xs = ⟨∑x: 0 ≤ x < #xs+ 1 : (x:xs).x⟩
{Logica}
sum.x:xs = ⟨∑x: x=0 v 1 ≤ x < #xs+ 1 : (x:xs).x⟩
{Particion de Rango}

sum.x:xs = ⟨∑x: x=0 : (x:xs).x⟩ + ⟨∑x: 1 ≤ x < #xs+ 1 : (x:xs).x⟩
{Rango unitario} {Cambio de variable x=x+1 }
sum.x:xs = (x:xs).0 + ⟨∑x: 1 ≤ x+1 < #xs+ 1 : (x:xs).x+1⟩
{Aritmetica}
sum.x:xs = (x:xs).0 + ⟨∑x: 0 ≤ x < #xs : (x:xs).x+1⟩
{caso base de .}{Caso recursivo de .}
sum.x:xs = x + ⟨∑x: 0 ≤ x < #xs : xs.x⟩
{HI}
sum.x:xs = x + sum.xs

3)

a) sum cuad.xs = ⟨∑ i : 0≤i<#xs : xs.i∗xs.i⟩
b) iga.v.xs = ⟨∀i : 0 ≤ i < #xs : xs.i = v⟩
c) fac.n = n!
Ayuda: Recordar que 0! = 1 y (n+1)! = (n+1)∗n! con n ∈ Nat

a) sum cuad.xs = ⟨∑ i : 0≤i<#xs : xs.i∗xs.i⟩

Caso Base: (xs= [ ])
sum cuad.[ ] = ⟨∑ i : 0 ≤ i < #[ ] : [ ].i ∗ [ ].i⟩
{Caso base de #}
sum cuad.[ ] = ⟨∑ i : 0 ≤ i < 0 : [ ].i ∗ [ ].i⟩
{Logica}
sum cuad.[ ] = ⟨∑ i : False : [ ].i ∗ [ ].i⟩
{Rango Vacío}
sum cuad.[ ] = 0

Hi: sum cuad.xs = ⟨∑ i : 0≤i<#xs : xs.i∗xs.i⟩

Caso Recursivo: (xs= x:xs)

sum cuad.(x:xs) = ⟨∑ i : 0 ≤ i < #(x:xs) : (x:xs).i ∗ (x:xs).i⟩
{Caso Recursivo de #}
sum cuad.(x:xs) = ⟨∑ i : 0 ≤ i < #xs+1 : (x:xs).i ∗ (x:xs).i⟩
{Logica}
sum cuad.(x:xs) = ⟨∑ i : i=0 v 1≤ i < #xs+1 : (x:xs).i ∗ (x:xs).i⟩
{Partición de rango }
sum cuad.(x:xs) = ⟨∑ i : i=0 : (x:xs).i ∗ (x:xs).i⟩
+
⟨∑ i : 1≤ i < #xs+1 : (x:xs).i ∗ (x:xs).i⟩
{Rango unitario, Cambio de variable (i -> i+1)}
sum cuad.(x:xs) = (x:xs).0 ∗ (x:xs).0
+
⟨∑ i : 1≤ i+1 < #xs+1 : (x:xs).i+1 ∗ (x:xs).i+1⟩

{Caso base de indexar, Aritmética}

sum cuad.(x:xs) = x ∗ x
+
⟨∑ i : 0 ≤ i < #xs : (x:xs).i+1 ∗ (x:xs).i+1⟩
{Caso recursivo de indexar}
sum cuad.(x:xs) = x ∗ x
+
⟨∑ i : 0 ≤ i < #xs : xs.i ∗ xs.i⟩
{Hi}
sum cuad.(x:xs) = x ∗ x
+
sum cuad.xs
PROGRAMABLE

```
sumCuad : : [Int] -> Int
sumCuad [ ] = 0
sumCuad (x:xs) = x * x + sumCuad xs
```

b) iga.v.xs = ⟨∀i : 0 ≤ i < #xs : xs.i = v⟩

iga.v.xs = ⟨∀i : 0 ≤ i < #xs : xs.i = v⟩

Caso base (xs=[ ])

iga.v.[ ] = ⟨∀i : 0 ≤ i < #[ ] : [ ].i = v⟩
{Caso base de #}
iga.v.[ ] = ⟨∀i : 0 ≤ i < 0 : [ ].i = v⟩
{Logica}
iga.v.[ ] = ⟨∀i : False : [ ].i = v⟩
{Rango Vacio}
iga.v.[ ] = True

Hi
iga.v.xs = ⟨∀i : 0 ≤ i < #xs : xs.i = v⟩

Caso Recursivo (xs= x:xs)
iga.v.(x:xs) = ⟨∀i : 0 ≤ i < #(x:xs) : (x:xs).i = v⟩
{Caso Recursivo de #}
iga.v.(x:xs) = ⟨∀i : 0 ≤ i < #xs+1 : (x:xs).i = v⟩
{Logica}
iga.v.(x:xs) = ⟨∀i : i=0 v 1≤ i < #xs+1 : (x:xs).i = v⟩
{Particion de Rango}
iga.v.(x:xs) = ⟨∀i : i=0 : (x:xs).i = v⟩ v ⟨∀i : 1≤ i < #xs+1 : (x:xs).i = v⟩

{Rango Unitario, Cambio de variable (i-> i+1) }
iga.v.(x:xs) = (x:xs).0 = v v ⟨∀i : 1≤ i+1 < #xs+1 : (x:xs).i+1 = v⟩
{Caso base de index, Caso recursivo de Index, Aritmetica restar 1 en cada termino de la desigualdad}
iga.v.(x:xs) = x = v v ⟨∀i : 0 ≤ i < #xs : xs.i = v⟩
{HI}
iga.v.(x:xs) = x = v v iga.v.xs
PROGRAMABLE

igaV : : Bool -> [Bool] -> Bool
igaV [ ] = True
igaV v (x:xs) = x = v v iga.v.xs

NO HAGO EL 4 Y 5 PORQUE NO ES DERIVAR

6)

b) sum par.n = ⟨∑ i : 0 ≤ i ≤ n ∧ par.i : i⟩
c) cuantos.v.xs = ⟨N i : 0 ≤ i < #xs : xs.i = v⟩
d) busca.v.xs = ⟨Min i : 0 ≤ i < #xs ∧ xs.i = v : i⟩

b) sumpar.n =⟨∑ i : 0 ≤ i ≤ n ∧ par.i : i⟩

Caso Base: (n=0)
sum par.0 = ⟨∑ i : 0 ≤ i ≤ 0 ∧ par.i : i⟩
{Logica}
sum par.0 = ⟨∑ i : i=0 ∧ par.i : i⟩
{Rango unitario}
sum par.0 = 0

Hi
sumpar.n =⟨∑ i : 0 ≤ i ≤ n ∧ par.i : i⟩

Caso Recursivo (n= n+1)

sumpar.n+1 =⟨∑ i : 0 ≤ i ≤ n+1 ∧ par.i : i⟩

{Logica}
sumpar.n+1 =⟨∑ i : (0 ≤ i ≤ n v i=n+1) ∧ par.i : i⟩
{Distributiva del v con el ^}
sumpar.n+1 =⟨∑ i : (0 ≤ i ≤ n ∧ par.i) v (par.i ^ i=n+1) : i⟩
{Particion de rango}
sumpar.n+1 =⟨∑ i : 0 ≤ i ≤ n ∧ par.i : i⟩ + ⟨∑ i : par.i ^ i=n+1 : i⟩
{Hi}
sumpar.n+1 =sumpar.n + ⟨∑ i : par.i ^ i=n+1 : i⟩
{Sustitucion de i=n+1}

sumpar.n+1 = sumpar.n + ⟨∑ i : par.n+1 : n+1⟩
{Casos}
sumpar.(n+1)= sumpar.n+ {n+1 si par.(n+1)
{0 en otro caso
PROGRAMABLE

c) cuantos.v.xs = ⟨N i : 0 ≤ i < #xs : xs.i = v⟩

Caso Base (xs= [ ])
cuantos.v.[ ] = ⟨N i : 0 ≤ i < #[ ] : [ ].i = v⟩
{Caso base de # }
cuantos.v.[ ] = ⟨N i : 0 ≤ i < 0 : [ ].i = v⟩
{ Logica y Rango vacio}
cuantos.v.[ ] = 0

Caso Recursivo (xs = x:xs)

cuantos.v.x:xs = ⟨N i : 0 ≤ i < #(x:xs) : (x:xs).i = v⟩
{Caso Recursivo de # }
cuantos.v.x:xs = ⟨N i : 0 ≤ i < #xs + 1 : (x:xs).i = v⟩
{Logica}
cuantos.v.x:xs = ⟨N i : i=0 v 1 ≤ i < #xs+1 : (x:xs).i = v⟩
{Particion de Rango}
cuantos.v.x:xs = ⟨N i : i=0 : (x:xs).i = v⟩ + ⟨N i : 1 ≤ i < #xs+1 : (x:xs).i = v⟩
{Rango unitario, Cambio de variable (i+1) y Aritmetica}
cuantos.v.x:xs = (x:xs).0 = v 1 + ⟨N i : 0 ≤ i < #xs : (x:xs).i+1 = v⟩
¬(x:xs).0 = v 0
{Caso base de indexar, Caso Recursivo de indexar}
cuantos.v.x:xs = x = v -> 1 + ⟨N i : 0 ≤ i < #xs : xs.i = v⟩
¬x = v -> 0
{HI}
cuantos.v.x:xs = x = v 1 + cuantos.v.xs
¬x = v 0

PROGRAMABLE

d) busca.v.xs = ⟨Min i : 0 ≤ i < #xs ∧ xs.i = v : i⟩

Caso Base (xs= [ ])
busca.v.[ ] = ⟨Min i : 0 ≤ i < #[ ] ∧ [ ].i = v : i⟩
{CB de #}
busca.v.[ ] = ⟨Min i : 0 ≤ i < 0 ∧ [ ].i = v : i⟩
{Logica}
busca.v.[ ] = ⟨Min i :False ∧ [ ].i = v : i⟩
{Absorbente de ^}
busca.v.[ ] = ⟨Min i :False : i⟩
{Rango Vacio}
busca.v.[ ] = + ∞ (es un infinito)

HI
busca.v.xs = ⟨Min i : 0 ≤ i < #xs ∧ xs.i = v : i⟩

Caso Recursivo (x:xs)
busca.v.x:xs = ⟨Min i : 0 ≤ i < #(x:xs) ∧ (x:xs).i = v : i⟩
{Caso Recursivo de #}
busca.v.x:xs = ⟨Min i : 0 ≤ i < #xs + 1 ∧ (x:xs).i = v : i⟩
{Logica}
busca.v.x:xs = ⟨Min i : (i=0 v 1 ≤ i < #xs + 1) ∧ (x:xs).i = v : i⟩
{Distributiva entre ^ v}
busca.v.x:xs = ⟨Min i : (i=0 ^ (x:xs).i = v) v ((x:xs).i =v ^ 1 ≤ i < #xs + 1) : i⟩
{Particion de Rango}
busca.v.x:xs = min ⟨Min i : i=0 ^ (x:xs).i = v : i⟩, ⟨Min i : (x:xs).i =v ^ 1 ≤ i < #xs + 1 : i⟩
{Reemplazo de i, Cambio de variable (i -> i+1)}
busca.v.x:xs = min ⟨Min i : (x:xs).0 = v : 0⟩, ⟨Min i : (x:xs).i+1 =v ^ 1 ≤ i+1 < #xs + 1 : i+1⟩
{Caso base de index, Caso recursivo de index}
busca.v.x:xs = min ⟨Min i : x = v : 0⟩,⟨Min i : xs.i = v ^ 1 ≤ i+1 < #xs + 1 : i+1⟩
{Aritmetica}
busca.v.x:xs = min ⟨Min i : x = v : 0⟩, ⟨Min i : xs.i = v ^ 0 ≤ i < #xs : i+1⟩
{Distributividad, Conmutatividad}
busca.v.x:xs = min ⟨Min i : x = v : 0⟩, ⟨Min i :0 ≤ i < #xs : i ^ xs.i = v⟩+1
{Casos , HI}
busca.v.x:xs = min { 0 si x=v, 1+ busca.v.xs si x/=v

Explicación

EXPLICACION
Cuando x=v

El mínimo índice es `0` (la cabeza de la lista).

No necesitamos mirar `xs`, porque ya encontramos la primera ocurrencia.

Cuando x/=v

Buscamos en la cola `xs` y **sumamos 1** al resultado (porque los índices de `xs` empiezan en `0` en la sublista, pero corresponden a `1, 2, ...` en la lista original `x:xs`).

Si `v` no está en `xs`, `busca.v.xs` da +∞

6 d, EJERCICIO DE MIERDA

8)

12) Generalización:

A) psum.xs = ⟨∀i : 0 ≤ i ≤ #xs : sum.(xs↑i) ≥ 0⟩

psum.xs = ⟨∀i : 0 ≤ i ≤ #xs : sum.(xs↑i) ≥ 0⟩
CB (xs = [ ])
psum.[ ] = ⟨∀i : 0 ≤ i ≤ #[ ] : sum.([ ]↑i) ≥ 0⟩
{Cb de length}
psum.[ ] = ⟨∀i : 0 ≤ i ≤ 0 : sum.([ ]↑i) ≥ 0⟩
{Logica}

psum.[ ] = ⟨∀i : i = 0 : sum.([ ]↑i) ≥ 0⟩
{Rango unitario}
psum.[ ] = sum.([ ]↑0) ≥ 0
{caso base de sum}
psum.[ ] = 0 ≥ 0
{Logica}
psum.[ ] = True

Caso Recursivo (xs= x:xs)

psum.(x:xs) = ⟨∀i : 0 ≤ i ≤ #(x:xs) : sum.((x:xs)↑i) ≥ 0⟩
{Caso Recursivo}
psum.(x:xs) = ⟨∀i : 0 ≤ i ≤ #xs+1 : sum.((x:xs)↑i) ≥ 0⟩
{Logica}
psum.(x:xs) = ⟨∀i : i = 0 v 1 ≤ i ≤ #xs+1 : sum.((x:xs)↑i) ≥ 0⟩
{Particion de Rango}
psum.(x:xs) = ⟨∀i : i = 0 : sum.((x:xs)↑i) ≥ 0⟩
^
⟨∀i : 1 ≤ i ≤ #xs+1 : sum.((x:xs)↑i) ≥ 0⟩
{Rango Unitario}{Cambio de Variable (i = i+1)}
psum.(x:xs) = sum.((x:xs)↑0) ≥ 0
^
⟨∀i : 1 ≤ i+1 ≤ #xs+1 : sum.((x:xs)↑i+1) ≥ 0⟩
{caso recursivo de tomar}{Aritmetica} {caso recursivo de tomar}
psum.(x:xs) = sum.([ ]) ≥ 0
^
⟨∀i : 0 ≤ i ≤ #xs : sum.(x:(xs↑i)) ≥ 0⟩
{Caso Base de sum}{caso recursivo de sum}
psum.(x:xs) = 0 ≥ 0
^
⟨∀i : 0 ≤ i ≤ #xs : x+ sum.(xs↑i) ≥ 0⟩
{Reflexividad} {Generalizacion}
psum.(x:xs) = True
^
⟨∀i : 0 ≤ i ≤ #xs : x+ sum.(xs↑i) ≥ 0⟩

Ahora vamos a realizar la generalizacion x va ser = n

psum.xs.n = ⟨∀i : 0 ≤ i ≤ #xs : n + sum.(xs↑i) ≥ 0⟩

Caso Base xs= [ ]

psum.[ ].n = ⟨∀i : 0 ≤ i ≤ #[ ] : n + sum.([ ]↑i) ≥ 0⟩
{Voy hacer todos los pasos en uno}
psum.[ ].n = n + sum.([ ]↑0) ≥ 0

{Caso base de Take y caso base de sum }
psum.xs.n = n + 0 ≥ 0

Caso Recursivo

psum.(x:xs).n = ⟨∀i : 0 ≤ i ≤ #(x:xs) : n + sum.((x:xs)↑i) ≥ 0⟩
{Caso Recursivo de #}
psum.(x:xs).n = ⟨∀i : 0 ≤ i ≤ #xs + 1 : n + sum.((x:xs)↑i) ≥ 0⟩
{Logica}
psum.(x:xs).n = ⟨∀i : i = 0 v 1 ≤ i ≤ #xs + 1 : n + sum.((x:xs)↑i) ≥ 0⟩
{Particion de Rango}
psum.(x:xs).n = ⟨∀i : i = 0 : n + sum.((x:xs)↑i) ≥ 0⟩
^
⟨∀ i : 1 ≤ i ≤ #xs + 1 : n + sum.((x:xs)↑i) ≥ 0⟩
{Rango Unitario}{Cambio de variable y aritmética}
psum.(x:xs).n = n + sum.((x:xs)↑0) ≥ 0
^
⟨∀i : 0 ≤ i ≤ #xs : n + sum.((x:xs)↑i+1) ≥ 0⟩
{Caso Recursivo de Take, Caso recursivo 2 de take}
psum.(x:xs).n = n + sum.([ ]) ≥ 0
^
⟨∀i : 0 ≤ i ≤ #xs : n + sum.(x:(xs↑i)) ≥ 0⟩
{caso base de sum}{Caso recursivo de sum}

psum.(x:xs).n = n ≥ 0
^
⟨∀i : 0 ≤ i ≤ #xs : (x+ n) + sum.(xs↑i) ≥ 0⟩
tomamos (x+n) como nuestra n generalizada y Hi
psum.n.(x:xs) =(n ≥ 0) ^ psum.xs.(n+x)

ahora finalizamos lo anterior con la nueva funcion creada

psum.(x:xs) = True
^
psum.xs.(n+x)